- *Replacement*—There's no default strategy to replace an object in a call stack; you have to find a way to do it yourself. There are dependency injection frameworks that provide some features to do this, but if any object refers directly to the old instance instead of through the level of indirection, you're in trouble. If you intend to change an object in place, you'd better make sure it works for multithreaded access.
- *Reboot*—Similar to replacement, getting an object back to an initial state is not automatically supported and takes another level of indirection you have to build. All the dependencies of the object must be reintroduced as well. If these dependencies also need to be rebooted (let's say the log processor can also throw a recoverable error), things can get complicated with regard to ordering.
- *Component lifecycle*—An object only exists after it's been constructed or is garbage-collected and removed from memory. Any other mechanism is something you have to build yourself.
- *Suspend*—When you catch an exception and throw it up the stack, the input data or some of its context is lost or unavailable. You have to build something to buffer the incoming calls while the error is unresolved. If the code is called from many threads, you need to add locks to prevent multiple exceptions from happening at the same time. And you have to find a way to store the associated input data to retry later.
- *Separation of concerns*—The exception-handling code is interwoven with the processing code and can't be defined independently of the processing code.

That doesn't look promising: getting everything to work correctly would be complex and a real pain. It looks like some fundamental